TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

OPERATING SYSTEMS LABORATORY 4

The xv6 Shell from the Outside

Student: Lucian-Adrian Gavril

Group: FAF-241

Coordinator: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

ABSTRACT 3
INTRODUCTION 4
1 SYSTEM CALL VERIFICATION AND SLEEP IMPLEMENTATION 5
1.1 System call declaration in user.h 5
1.2 Implementation of user/sleep.c 5
1.3 Runtime execution in xv6 6
2 COMPARATIVE ARCHITECTURE: BASH VERSUS XV6 SHELL 8
2.1 Systematic comparison matrix 8
2.2 Shell command captures 9
2.3 Path resolution and directory navigation 10
2.4 Answers to Part 2 Observe questions 11
3 ARGUMENT TOKENIZATION: WHO EXPANDS WILDCARDS? 12
3.1 Implementation of argument vector inspectors 12
3.2 Experimental comparison across shells 12
3.3 Answers to Part 3 Observe questions 13
4 ARCHITECTURE OF USER/SH.C 15
4.1 Command representations and syntax nodes 15
4.2 Recursive command execution in runcmd() 15
4.3 Built-in cd handling and process boundaries 16
4.4 Answers to Part 4 Observe questions 17
CONCLUSIONS 18
REFERENCES 19

ABSTRACT

This report examines how the xv6 shell turns command text into process execution. It explains file redirection, pipelines, command sequencing, the shell command tree in `user/sh.c`, and why the built-in `cd` must run in the parent process. It also inspects the `sleep` and `args` utilities and compares their argument handling with GNU Bash. A 9 October 2026 QEMU run verified the no-argument `sleep` message and `sleep 20`, which returned to the prompt in 1.96 seconds. A 10 October QEMU run verified all four requested `args` cases. The screenshots are genuine terminal captures from Dev1 Debian GNU/Linux 13; they do not establish that a course-assigned laptop or VM was used.

Keywords: operating systems, xv6, shell architecture, command line, process hierarchy, argument passing, riscv

INTRODUCTION

Operating system shells mediate the boundary between user input and kernel process services. A shell parses commands, configures file descriptors, starts processes, creates pipelines, and invokes `exec()` [1, 3].

Laboratory 4 examines the xv6 shell (`user/sh.c`) alongside GNU Bash to identify core shell primitives and the user-space features omitted by xv6. The host did not have a globally installed RISC-V toolchain or QEMU, and `sudo -n true` returned exit code 1 with `sudo: a password is required`. Official Debian packages were therefore extracted under a temporary `/tmp` prefix without installing host packages. On 9 October, `make qemu` booted a temporary copy of the course tree. A second focused `make qemu` run on 10 October tested all four argument patterns. The figures below show genuine Bash and QEMU captures from the Dev1 Debian GNU/Linux 13 host; they do not establish that the course-assigned laptop or VM was used.

The investigation is divided into four parts:

a) **System Call Verification and Sleep Implementation**: Validating the kernel system call declaration in `user/user.h`, confirming proper argument parsing and tick delays in `user/sleep.c`, and registering the utility in the xv6 build system.

b) **Comparative Behavioral Analysis**: Testing identical syntax patterns in Bash and xv6 across I/O redirection, pipelines, list grouping, sequential execution, background jobs, globbing, environment variables, quotation semantics, path searching, and command-line editing.

c) **Argument Vector Inspection**: Authoring dual implementations of an argument printer (`args.c` for xv6 and `args_linux.c` for Linux) to determine who expands wildcards and strips quotation marks.

d) **Source Code Architecture of `user/sh.c`**: Examining the tagged command structures, recursive execution in `runcmd()`, and the explicit parent-process handling of the `cd` command.

1 SYSTEM CALL VERIFICATION AND SLEEP IMPLEMENTATION

1.1 System call declaration in user.h

In xv6, system calls available to user programs are prototyped in `user/user.h`. Because system calls represent kernel traps triggered by assembly stubs in `usys.S`, the function signatures in `user.h` must match the dispatch tables in `kernel/syscall.c`.

The source capture shows the system-call declaration in the copied header (Figure 1.1) [1].

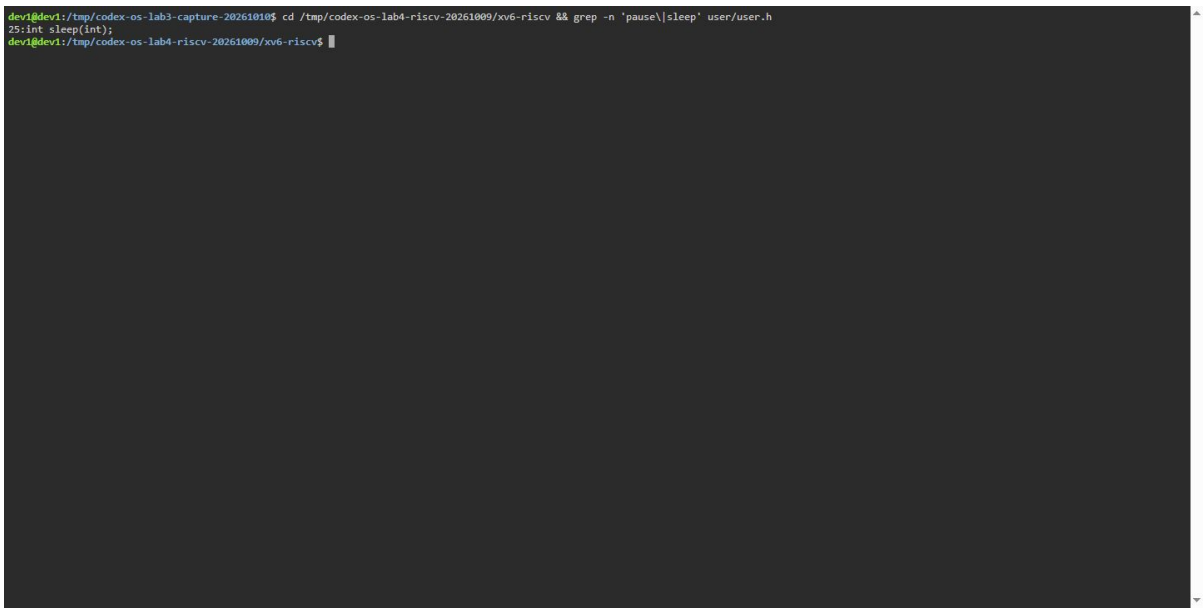


Figure 1.1 – Terminal capture, Dev1 Debian GNU/Linux 13, 10 October 2026: System call declaration verified within user/user.h.

1.2 Implementation of user/sleep.c

The `sleep` utility serves as an interface between command-line arguments and the kernel timer tick scheduler. The utility verifies that exactly one argument is supplied (`argc == 2`). If the parameter count is invalid, an error line is emitted to file descriptor 2 (stderr), and the process exits with status 1. If valid, `atoi()` parses the argument string into an integer and passes it to the system call (Figure 1.2).

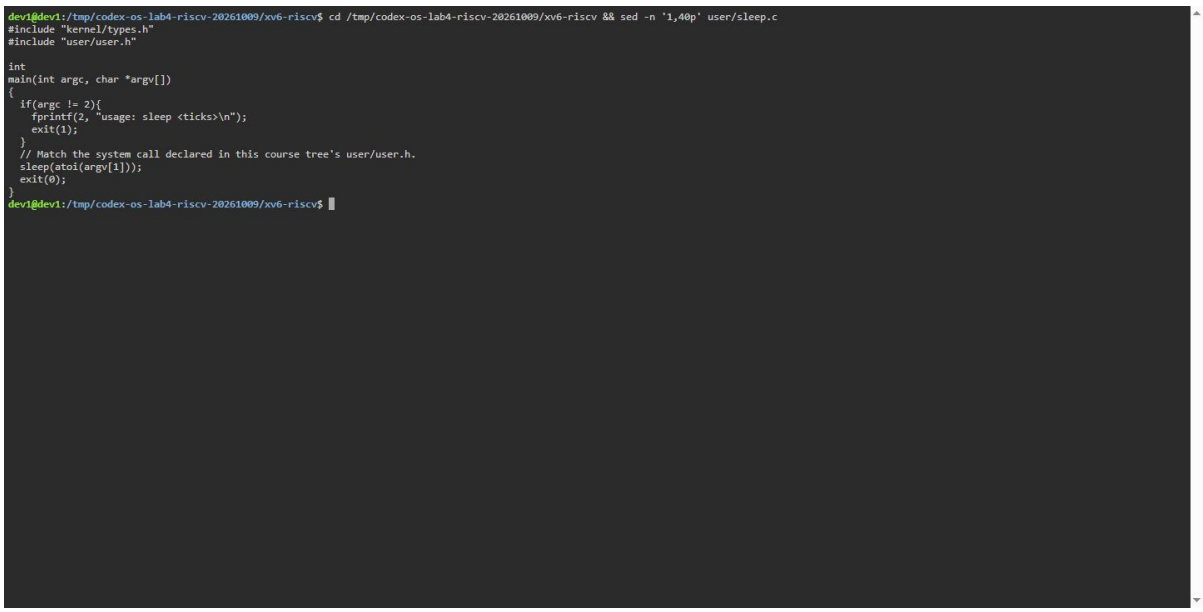


Figure 1.2 – Terminal capture, Dev1 Debian GNU/Linux 13, 10 October 2026: Implementation of user/sleep.c with input validation.

1.3 Runtime execution in xv6

The 9 October QEMU run returned to the prompt after `sleep 20` in 1.96 seconds of measured wall time. It also printed the no-argument usage message and checked two `args` cases. The following excerpt preserves that earlier serial transcript; the 10 October four-case run is shown in Figure 3.2:

```
init: starting sh
$ sleep
usage: sleep <ticks>
$ sleep 20
$ args one two three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ echo LAB4_SLEEP_DONE
LAB4_SLEEP_DONE
$
```

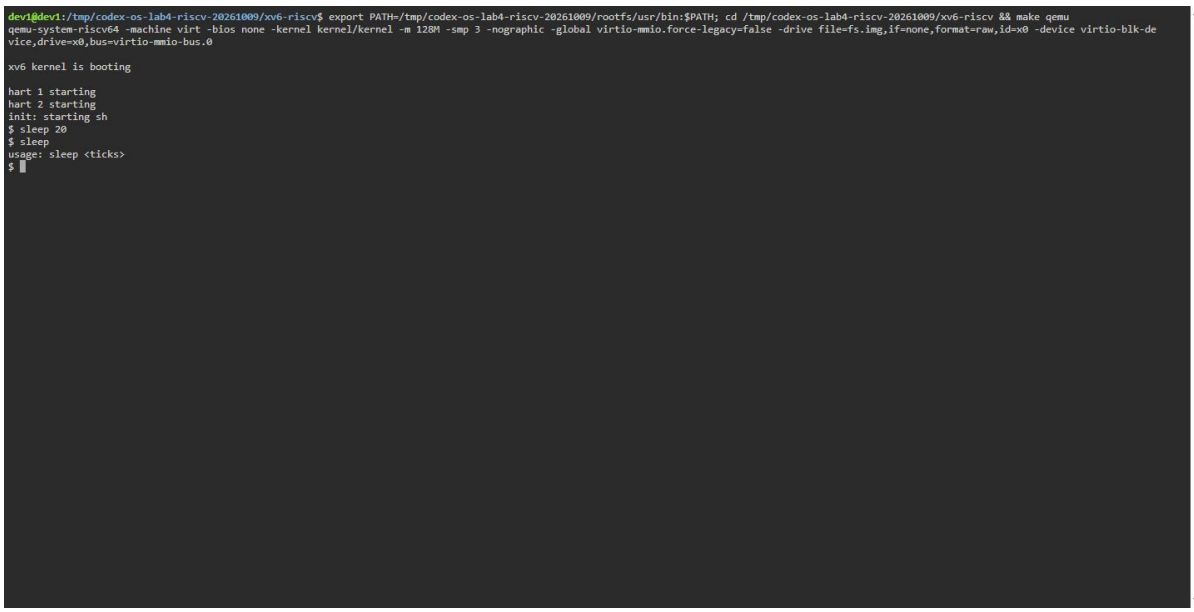


Figure 1.3 – Terminal capture, Dev1 Debian GNU/Linux 13, 10 October 2026: xv6 boot, `sleep 20`, and the no-argument usage message.

2 COMPARATIVE ARCHITECTURE: BASH VERSUS XV6 SHELL

2.1 Systematic comparison matrix

The matrix records observed output for nine shell commands and separate keyboard-behavior checks. The Bash and xv6 terminal captures show command rows 1–9; Bash history, completion, and reverse-search behavior and the xv6 line-editing behavior were checked separately. Explanations are based on the course tree and shell source [1, 4, 3].

Test Paradigm	Bash Output	xv6 Output	Architectural Explanation
Redirection `echo hello > f ; cat < f`	`hello`	`hello`	Both implement `open()` and `dup()` redirection via file descriptors.
Pipe `ls | grep READ`	`README`	`README`	Both create kernel pipe buffers and coordinate reader/writer via `pipe()`.
Grouping `(echo a; echo b) | wc`	`2 2 4`	`2 2 4`	Both shells accept the tested grouped command in a pipeline; `wc` reports lines, words, and bytes.
Sequence `echo one ; echo two`	`one` `two`	`one` `two`	Both implement sequential command execution via `LIST` nodes separated by `;`.
Background `echo bg &`	`bg` (background)	`bg`	Both support asynchronous child execution by omitting `wait()`.
Globbing `ls *`	Lists all entries	`ls: cannot open *`	Bash performs pathname expansion; xv6 passes literal wildcard characters.
Variables `echo $HOME`	`/home/dev1`	`$HOME`	Bash expands the host variable; xv6 prints the token literally.
Quotes `echo "a b"`	`a b` (single arg)	`"a b"` (literal)	Bash removes quotes and protects whitespace; xv6 splits strictly on spaces.
PATH Lookup `mkdir d ; cd d ; ls ..`	Lists parent-directory entries	`exec cd failed`; root entries listed	Bash runs `cd` in the list and uses `$PATH`; xv6 tries `exec cd`, then runs `ls ..` from `/`.
Line Editing `Arrows, Tab, ^R`	Up: history; Tab: completion; Ctrl+R: reverse search	Up/Ctrl+R: no history or search; blank-prompt Tab: no completion list	Bash behavior was checked interactively. The focused QEMU check found no Up-arrow recall or Ctrl+R search. Blank-prompt Tab produced no completion list; path-prefix completion was not tested. Source inspection shows no such handler in the xv6 console.

Table 2.1 – Command results for rows 1–9; line-editing behavior was checked separately in Bash and QEMU.

2.2 Shell command captures

The Bash capture shows the nine shell commands run on Dev1 Debian GNU/Linux 13. Up-arrow, Tab, and Ctrl+R behavior was checked separately in an interactive Bash session.

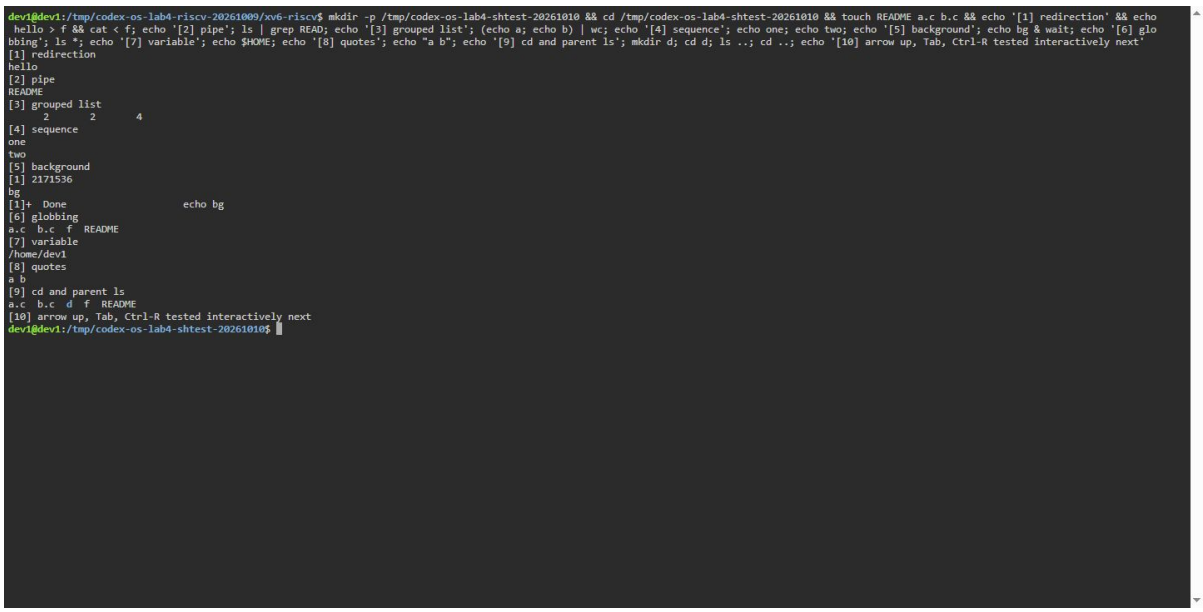


Figure 2.1 – Terminal capture, Dev1 Debian GNU/Linux 13, 10 October 2026: Bash results for command rows 1 through 9.

The xv6 capture shows nine shell commands. An interaction tested Up, Tab, and Ctrl+R in QEMU: Up and Ctrl+R showed no history or search; Tab at an empty prompt produced no completion list. The console source in `kernel/console.c`, lines 140–177, handles Ctrl+P, Ctrl+U, Ctrl+H, and Delete. Path-prefix completion was not tested.

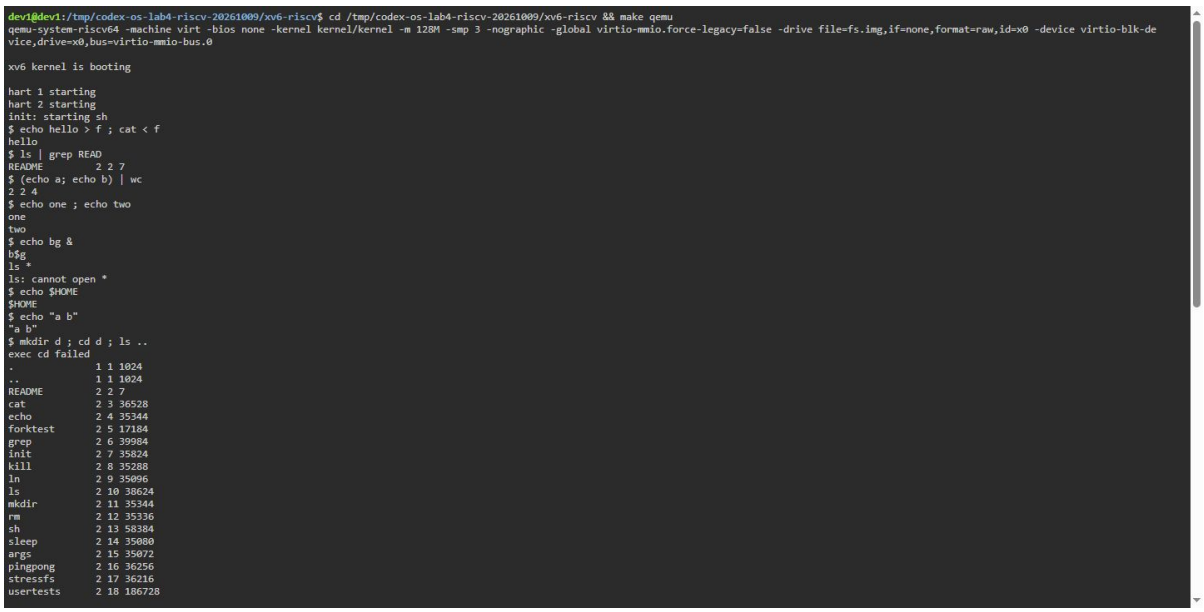


Figure 2.2 – Terminal capture, Dev1 Debian GNU/Linux 13, 10 October 2026: xv6 QEMU results for comparison rows 1 through 9.

2.3 Path resolution and directory navigation

The QEMU capture shows that `cd d` succeeds at a prompt, a later unqualified `ls` fails with `exec ls failed`, and a relative executable path works (Figure 2.3).

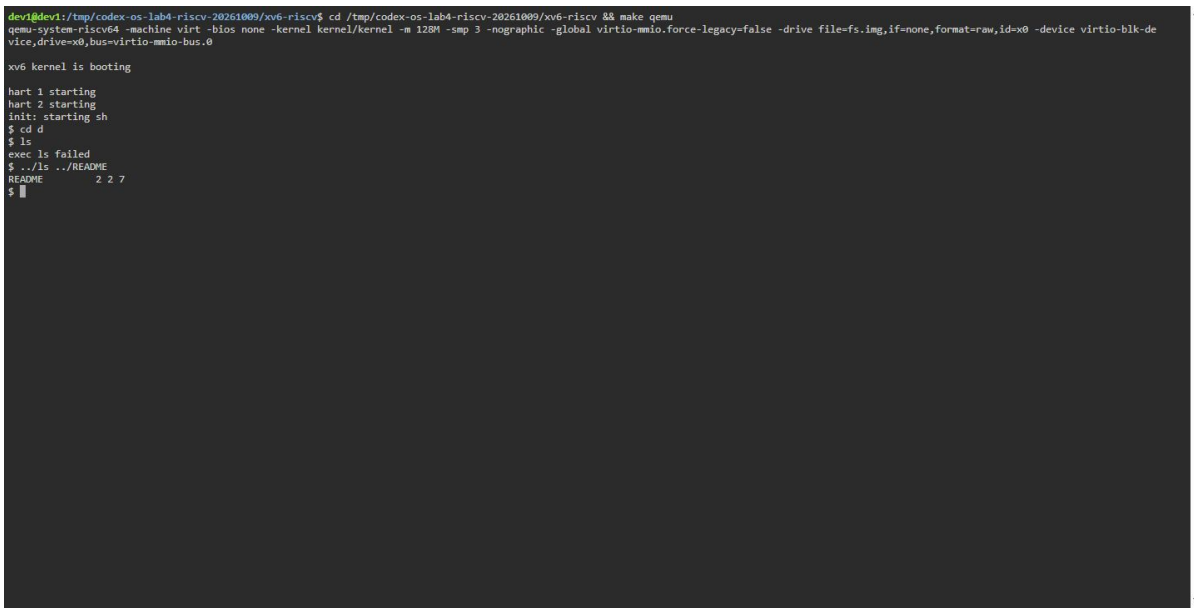


Figure 2.3 – Terminal capture, Dev1 Debian GNU/Linux 13, 10 October 2026: path lookup after changing directories in xv6.

2.4 Answers to Part 2 Observe questions

a) **List the features both shells have, and the ones only Bash has.**

Features in both: Direct command execution, file descriptor redirection (<, >), pipelines (|), parenthesized command grouping, sequential execution (;), and asynchronous background execution (&). The grouped xv6 command in the test returned `2 2 4`.

Features only in Bash: Globbing (*, ?, `[...]`), environment variables (`$VAR`), quote processing (`"..."`, `'...'`), conditional operators (&&, ||), automatic `PATH` search, history search, and tab completion.

b) **In xv6, `ls` stops working after `cd d`. Why? Find a way to run `ls` from inside `d` anyway.**

At the top-level prompt, `cd d` succeeds. The xv6 shell has no `PATH` lookup, so `ls` is resolved relative to `d` and fails. Figure 2.3 shows the working relative path `../ls ../README`, which explicitly invokes the executable in the parent directory.

c) **Which missing feature would you miss most in daily work, and which would be easiest to add?**

Most missed: `PATH` directory resolution and tab completion/history. Without `PATH`, changing directories renders all core utilities inaccessible unless typed with full paths.

Easiest to add: A fallback path search in `user/sh.c`. If `exec(cmd, argv)` returns failure, prepend `"/"` or `"/bin/"` to `cmd` and re-execute before returning an error to the user.

3 ARGUMENT TOKENIZATION: WHO EXPANDS WILDCARDS?

3.1 Implementation of argument vector inspectors

To demonstrate precisely when and where argument expansions occur, identical inspection utilities were authored for xv6 (`user/args.c`) and Linux (`args_linux.c`). Each prints `argc` and prints each `argv[i]` string enclosed in brackets.

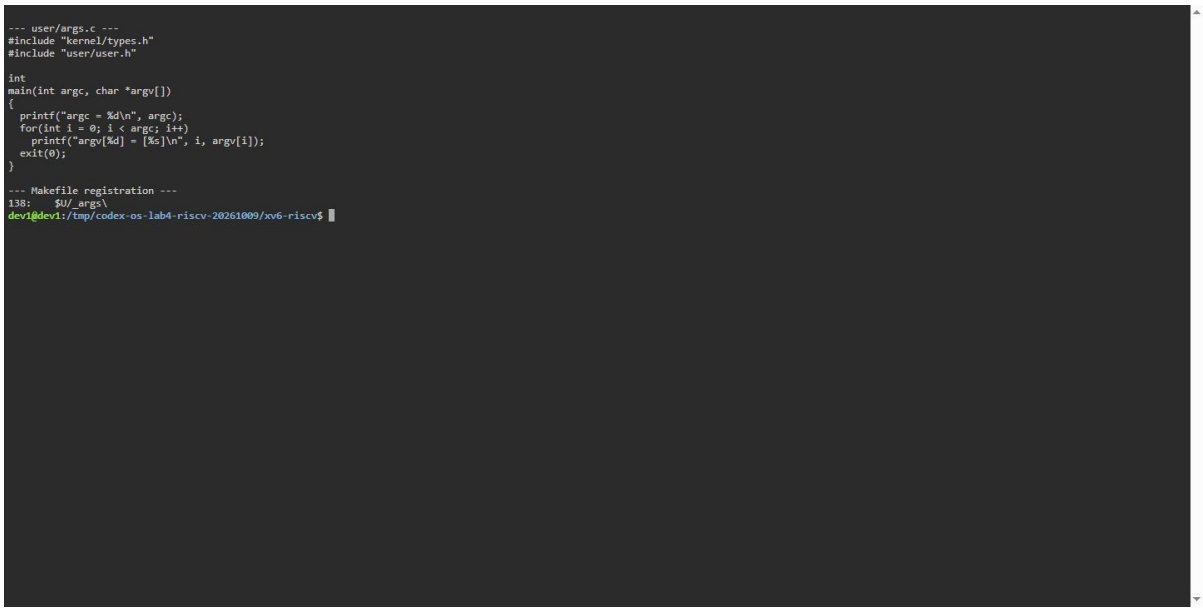


Figure 3.1 – Terminal capture, Dev1 Debian GNU/Linux 13, 10 October 2026: Source of user/args.c and registration within xv6 Makefile UPROGS.

The copied `Makefile` registers both `$U/_sleep` and `$U/_args` in `UPROGS`, so the build includes the utilities in `fs.img`. The capture shows the source and registration lines; the 9 and 10 October QEMU runs built and booted this temporary tree.

3.2 Experimental comparison across shells

Both implementations were run with all four requested argument patterns. The Linux capture used a working directory containing `a.c`, `b.c`, and `args_linux.c`; the xv6 capture shows the arguments received in QEMU:

1. `args one two three`
2. `args *.c "x y"`
3. `args $HOME`
4. `args 'single quoted'`

Figure 3.2 shows the actual output from all four xv6 invocations.

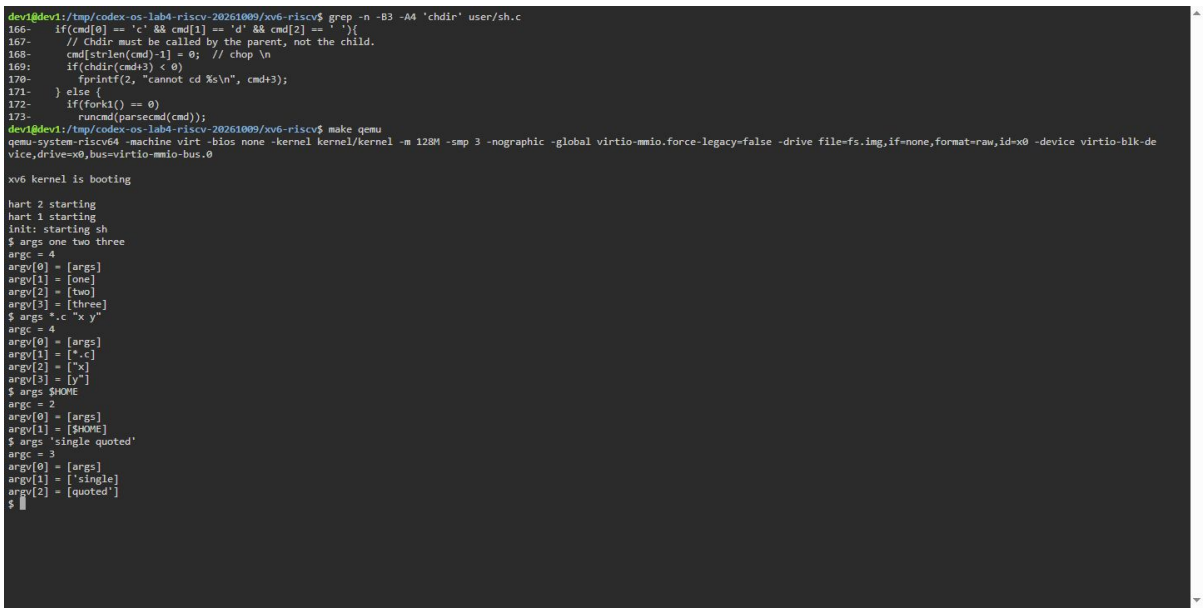


Figure 3.2 – Terminal capture, Dev1 Debian GNU/Linux 13, 10 October 2026: xv6 output for all four `args` patterns.

Figure 3.3 shows the actual Linux Bash output from the same four invocations.

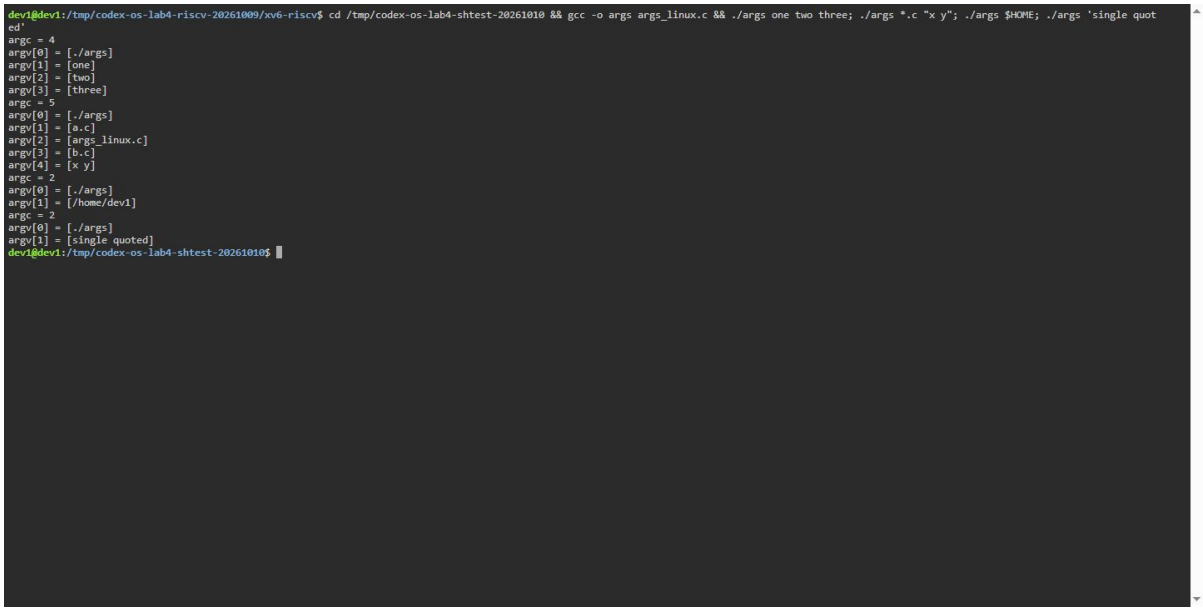


Figure 3.3 – Terminal capture, Dev1 Debian GNU/Linux 13, 10 October 2026: Linux Bash output for all four `args` patterns.

3.3 Answers to Part 3 Observe questions

a) **Paste both outputs for `args *.c "x y"`. How many arguments did each program receive?**

In Linux Bash:

```
argc = 5
argv[0] = [./args]
argv[1] = [a.c]
```

```
argv[2] = [args_linux.c]
argv[3] = [b.c]
argv[4] = [x y]
```

In xv6 Shell:

```
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
```

The Linux program received 5 arguments because the wildcard matched three source files and the quoted phrase remained one argument. xv6 received 4 arguments: it preserved `*.c` literally and split `"x y"` into two strings with quotes intact.

b) **Who turns `*.c` into file names and removes quotes: the program, the kernel, or the shell?**
The shell. The shell performs lexical scanning, globbing, variable expansion, and quote stripping before invoking `exec()`. The kernel merely accepts an array of already-expanded null-terminated strings and copies them to the new process stack. The user program receives the strings exactly as prepared by the shell.

c) **What happened to the extra spaces in `one two three`?**
The shell treats spaces as argument separators (delimiters). Multiple contiguous spaces are consumed by the lexer during tokenization, so only the non-whitespace tokens (`"one"`, `"two"`, `"three"`) are inserted into the argument array.

4 ARCHITECTURE OF USER/SH.C

4.1 Command representations and syntax nodes

The xv6 shell represents commands using tagged C structures. Five command types are defined on lines 8–12 of `user/sh.c` (Figure 4.1):

- `EXEC (1)`: Basic command execution; no operator.
- `REDIR (2)`: Redirection of I/O; produced by < and >.
- `PIPE (3)`: Inter-process communication; produced by `|`.
- `LIST (4)`: Sequential commands; produced by `;`.
- `BACK (5)`: Asynchronous background execution; produced by `&`.

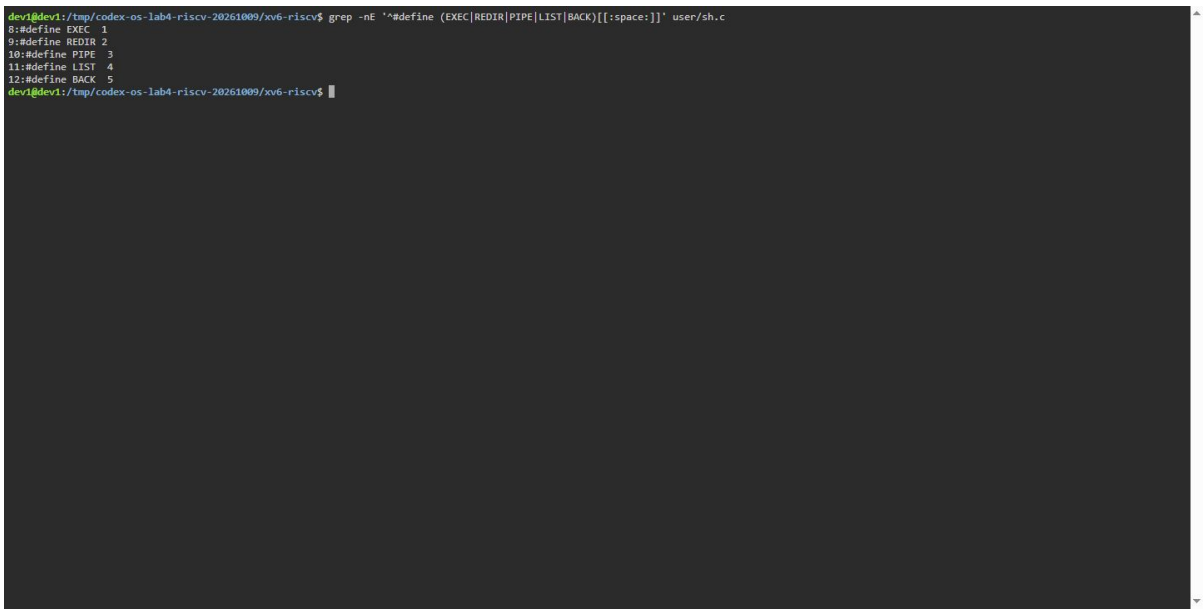


Figure 4.1 – Terminal capture, Dev1 Debian GNU/Linux 13, 10 October 2026: Command types defined in user/sh.c.

4.2 Recursive command execution in runcmd()

When a command line is parsed, `parsecmd()` constructs an abstract syntax tree composed of these structures. The child process then invokes `runcmd()` [1] (Figure 4.2).

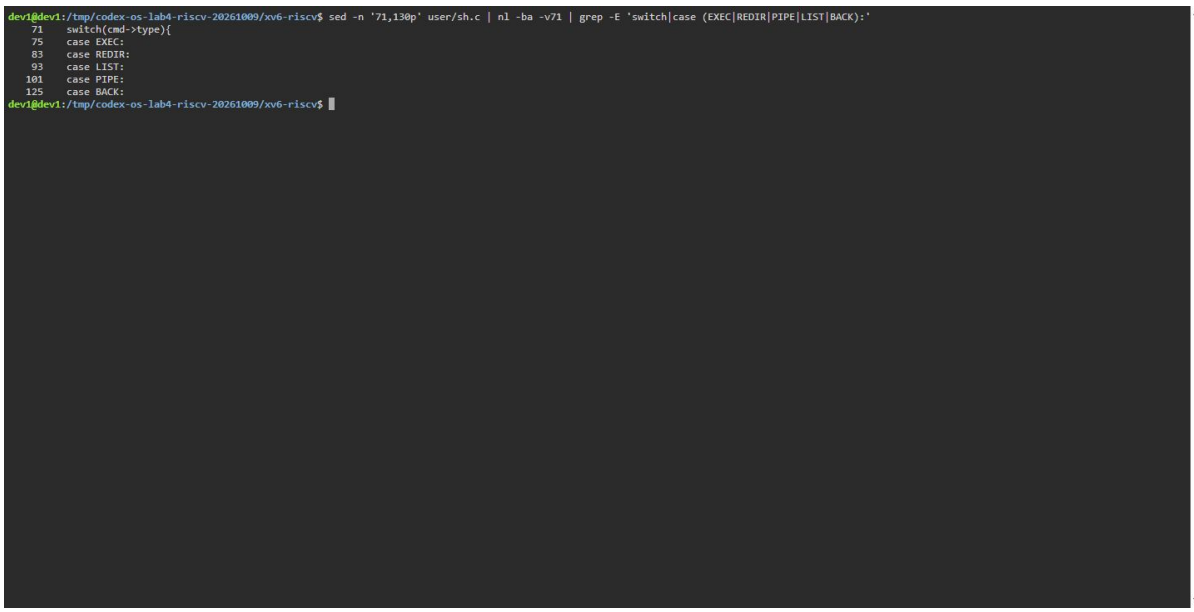


Figure 4.2 – Terminal capture, Dev1 Debian GNU/Linux 13, 10 October 2026: Dispatch switch inside runcmd() handling syntax tree execution.

The execution logic handles each node type:

- For `EXEC`, it invokes `exec(ecmd->argv[0], ecmd->argv)`.
- For `REDIR`, it closes the targeted descriptor, opens the named file with the requested flags, and recursively evaluates the inner command.
- For `PIPE`, it calls `pipe()`, forks two sub-processes, rewires descriptors 0 and 1 using `dup()`, closes pipe ends, and waits for both children to terminate.
- For `LIST`, it forks a child to run the left command, waits for it, and then runs the right command.
- For `BACK`, it forks a child to run the command and immediately returns without waiting.

4.3 Built-in cd handling and process boundaries

Lines 166–170 of `user/sh.c` reveal that `cd` is explicitly intercepted in the main shell loop before any call to `fork()` (Figure 4.3).

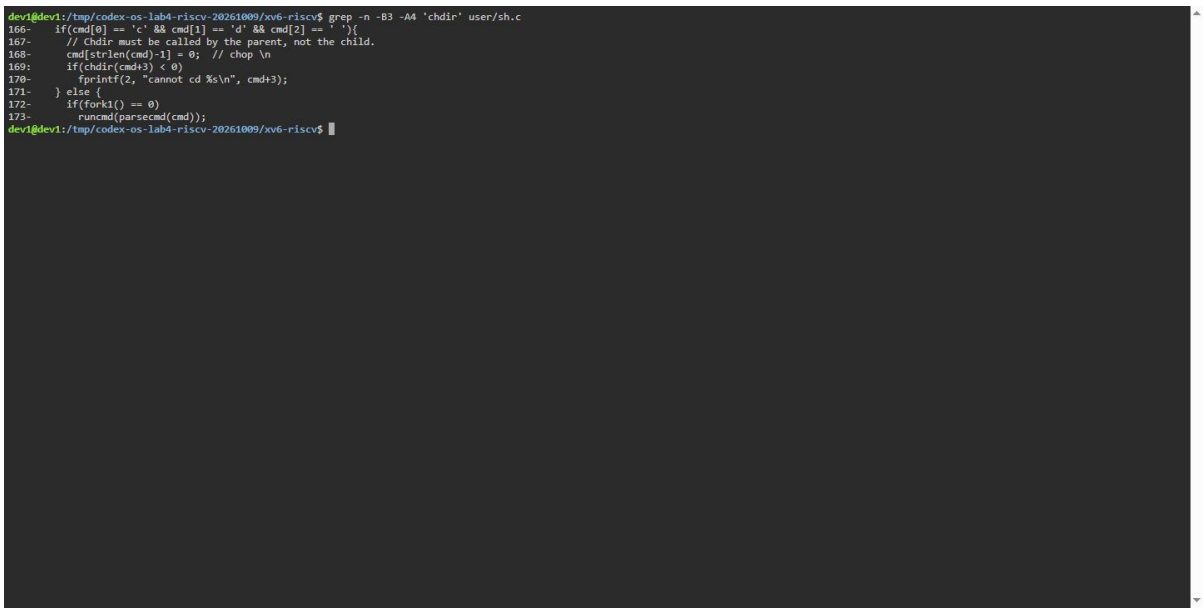


Figure 4.3 – Terminal capture, Dev1 Debian GNU/Linux 13, 10 October 2026: Parent-process handling of cd built-in in user/sh.c.

4.4 Answers to Part 4 Observe questions

a) **Name the five kinds of command the xv6 shell knows and the character that produces each.**
`EXEC` (basic command, no special character), `REDIR` (produced by < and >), `PIPE` (produced by `|`), `LIST` (produced by `;`), and `BACK` (produced by `&`).

b) **Where is `cd` handled, and why is it not run as a separate program?**
`cd` is handled in `main()` of `user/sh.c` prior to the `fork()` call. It cannot be run as a separate program because each child process operates in its own isolated address space and process descriptor table. If `cd` were an external program, only the child process would change its directory; upon child exit, the parent shell would remain in the original directory. Therefore, `chdir()` must execute directly in the parent shell process.

c) **Search `sh.c` for any code that handles `*` or `$`. What does that explain about Part 2?**
Searching `sh.c` reveals zero code handling `*` or `$`. Neither globbing nor variable expansion logic exists in the parser. This explains why `echo $HOME` and `ls *` print literal characters in xv6: the shell treats them as ordinary alphanumeric tokens and passes them uninspected to `exec()`.

CONCLUSIONS

The captures distinguish Bash expansion from xv6 token splitting, while the source figures identify the xv6 command types and parent-process handling of `cd`. A focused QEMU check found no Up-arrow recall or Ctrl+R search, and Tab at an empty prompt produced no completion list. Source inspection confirms that the xv6 console has no history, completion, or reverse-search handler; path-prefix completion was not tested.

REFERENCES

1. Cox, R., Kaashoek, M. F., & Morris, R. (2023). *xv6: a simple, Unix-like teaching operating system*. MIT CSAIL.

2. Patterson, D. A., & Hennessy, J. L. (2017). *Computer Organization and Design RISC-V Edition*. Morgan Kaufmann.

3. Love, R. (2013). *Linux System Programming: Talking Directly to the Kernel and C Library* (2nd ed.). O'Reilly Media.

4. Bach, M. J. (1986). *The Design of the UNIX Operating System*. Prentice-Hall.